A trace semantics for the process calculus of ProVerif

Artie Khovanov with lots of assistance from Claude Fable

ProVerif is a tool for reasoning about cryptographic security in a symbolic model with a Dolev-Yao adversary. Although it uses resolution of Horn clauses to find attacks, it is backed theoretically by a process calculus, which is a variant of the applied pi calculus. We use the specification of ProVerif's semantics given in Blanchet 16 [1].

We formalise the ProVerif process calculus in Lean. Rather than copying the specification, we use a shallow translation in order to make security proofs simpler down the line. This means that we make use of existing Lean syntax and constructs where possible. In particular, some invariants are enforced only by what terms can arise from translating a valid ProVerif program. We also deviate from the calculus when sticking exactly to the specification would complicate the design.

We use a trace-based semantics to encode the possible outcomes of parallel processes communicating on a network. The attacker is modelled as a set of traces satisfying informational constraints. Our trace semantics are good enough to express secrecy and simple ProVerif queries relating events. However, it is strictly less expressive than the formal operational semantics of ProVerif; in particular, we cannot express observational equivalence.

We give a mathematical account of the Lean development, and of the formal translation from terms in ProVerif's process calculus to our traces. Everything has been formalised in Lean.

Section 1 presents the trace semantics; Section 2 the calculus of ProVerif, its syntax and the semantics of the survey; and Section 3 the translation of the calculus into the trace semantics and the theorems relating the two. In the development these are the folders `TraceSemantics` and `Specification`, the translation being written twice: as the denotation in `Specification`, which the theorems are about, and as the program `Translation`, which turns a ProVerif model into Lean source over the trace semantics.

1 The trace semantics

1.1 Messages

Definition 1.1 (Signature, terms)**.** A *signature* is a family $\Sigma = (\Sigma_n)_{n\in\mathbb{N}}$ of sets; $f \in \Sigma_n$ is a constructor of arity n. The set $\mathcal{T} = \mathcal{T}(\Sigma)$ of *raw terms* is generated by

$$t ::= \nu_k \ (k \in \mathbb{N}) \ \mid \ f(t_1, \ldots, t_n) \ (f \in \Sigma_n).$$

A *substitution* is a map $\sigma : \mathbb{N} \to \mathcal{T}$; $t\sigma$ replaces each ν_k in t by $\sigma(k)$.

Definition 1.2 (Theory)**.** A *theory* over Σ is a pair (E, Fr) with $E \subseteq \mathcal{T} \times \mathcal{T}$ and $\mathrm{Fr} \subseteq \bigcup_n \Sigma_n$ such that

(H) for every $(l, r) \in E$, both l and r are of the form $f(\dots)$ with $f \notin \mathrm{Fr}$.

The relation $\equiv_E$ is the least congruence on $\mathcal{T}$ containing $l\sigma \equiv_E r\sigma$ for all $(l, r) \in E$ and all substitutions σ. The set of *messages* is $\mathcal{M} = \mathcal{T}/\equiv_E$, with $[t]$ the class of t. Constructors act on $\mathcal{M}$ by $f([t_1], \dots, [t_n]) = [f(t_1, \dots, t_n)]$, and ν_k also denotes $[\nu_k]$.

A Σ-algebra A *satisfies* E if $l\sigma_A = r\sigma_A$ for every $(l, r) \in E$ and every valuation $\sigma_A : \mathbb{N} \to A$, where $t\sigma_A$ is the evaluation of t in A.

Proposition 1.3. *$\mathcal{M}$ satisfies E. For every Σ-algebra A satisfying E and every $v : \mathbb{N} \to A$ there is a unique homomorphism $\mathcal{M} \to A$ sending ν_k to $v(k)$.*

Definition 1.4 (View)**.** Let $\mathcal{V} = \mathbb{N} \uplus \{f(m_1, \dots, m_n) \mid f \in \mathrm{Fr} \cap \Sigma_n,\ m_i \in \mathcal{M}\} \uplus \{\bot\}$. Define $\mathrm{view}_0 : \mathcal{T} \to \mathcal{V}$ by

$$\mathrm{view}_0(\nu_k) = k, \qquad \mathrm{view}_0(f(t_1, \dots, t_n)) = \begin{cases} f([t_1], \dots, [t_n]) & f \in \mathrm{Fr}, \\ \bot & f \notin \mathrm{Fr}. \end{cases}$$

Proposition 1.5. *The following hold.*

1. *$s \equiv_E t$ implies $\mathrm{view}_0(s) = \mathrm{view}_0(t)$; hence view_0 induces $\mathrm{view} : \mathcal{M} \to \mathcal{V}$.*
2. *$\mathrm{view}(f(m_1, \dots, m_n)) = f(m_1, \dots, m_n)$ if $f \in \mathrm{Fr}$, and $\bot$ otherwise.*
3. *If $\mathrm{view}(m) = f(m_1, \dots, m_n)$ then $m = f(m_1, \dots, m_n)$.*
4. *$k \mapsto \nu_k$ is injective $\mathbb{N} \to \mathcal{M}$.*

Definition 1.6 (May-fail messages, lifting)**.** Let $\mathcal{M}_\bot = \mathcal{M} \uplus \{\bot\}$. For $g : \mathcal{M}^n \to \mathcal{M}_\bot$ let $g^\uparrow : \mathcal{M}_\bot^n \to \mathcal{M}_\bot$ be $g^\uparrow(\vec{u}) = g(\vec{u})$ if every $u_i \in \mathcal{M}$ and $\bot$ otherwise; $f^\uparrow$ for $f \in \Sigma_n$ is the lift of $f : \mathcal{M}^n \to \mathcal{M} \subseteq \mathcal{M}_\bot$. A *destructor* of arity n is a function $\mathcal{M}_\bot^n \to \mathcal{M}_\bot$.

Definition 1.7 (Data symbols, standard operations)**.** The *data symbols* are a tuple constructor tuple_n of each arity $n \in \mathbb{N}$ and the constants true and false. A signature *with data* is $\Sigma = \Sigma_0 \uplus \mathrm{Data}$ for a signature Σ_0 of *declared* symbols, and a theory (E, Fr) over Σ_0 is read over Σ with the data symbols free. We write $(m_1, \dots, m_n)$ for $\mathsf{tuple}_n(m_1, \dots, m_n)$. The *standard operations* Std are the lifts $\mathsf{tuple}_n^\uparrow$, the projections $\pi_i^n : \mathcal{M}_\bot \to \mathcal{M}_\bot$ with $\pi_i^n(u) = m_i$ if $u = (m_1, \dots, m_n)$ and $\bot$ otherwise, well defined by Proposition 1.5, and the constants true and false.

Definition 1.8 (Public operations, deduction)**.** Let $\mathrm{Pub} \subseteq \bigcup_n \Sigma_n$ and let Dt be a set of destructors. Set $\mathrm{Ops} = \{f^\uparrow \mid f \in \mathrm{Pub}\} \cup \mathrm{Dt} \cup \mathrm{Std}$. A set $S \subseteq \mathcal{M}$ is Ops-*closed* if for every $o \in \mathrm{Ops}$ of arity n and every $\vec{u} \in (S \uplus \{\bot\})^n$, $o(\vec{u}) \in \mathcal{M}$ implies $o(\vec{u}) \in S$. For $K \subseteq \mathcal{M}$,

$$\langle K \rangle = \bigcap \{S \supseteq K \mid S \text{ is Ops-closed}\}, \qquad K \vdash m \iff m \in \langle K \rangle.$$

Proposition 1.9. *$\langle\cdot\rangle$ is a closure operator on $\mathcal{P}(\mathcal{M})$, $\langle K \rangle$ is* Ops*-closed, and $K \vdash m_1, \dots, K \vdash m_n$ imply $K \vdash f(m_1, \dots, m_n)$ for $f \in \mathrm{Pub} \cap \Sigma_n$ or f a data symbol. In particular $\emptyset \vdash a$ for $a \in \mathrm{Pub} \cap \Sigma_0$.*

Remark 1.10. In the development the data symbols are added to a signature and a theory by withData, and the standard operations are the class StdOps on a type of messages, which the messages of a theory with data instantiate; Ops is the class Attacker, and Attacker.ofDecls builds it from Pub and Dt.

1.2 Traces and processes

Definition 1.11 (Actions, traces)**.**

$$\mathrm{Act} = \{\mathsf{out}(c,m),\ \mathsf{in}(c,m),\ \mathsf{comm}(c,m),\ \mathsf{new}(k),\ \mathsf{ev}(m) \mid c,m \in \mathcal{M},\ k \in \mathbb{N}\}.$$

A *trace* is a word $t \in \mathrm{Act}^*$; ε is the empty trace and $l \le t$ means l is a prefix of t. The actions $\mathsf{out}(c,m)$ and $\mathsf{in}(c,m)$ are *unfinished*; $\mathrm{Closed}(t)$ holds if no action of t is unfinished. $\mathrm{nonces}(t) \in \mathbb{N}^*$ is the image of t under the monoid homomorphism with $\mathsf{new}(k) \mapsto k$ and every other action $\mapsto \varepsilon$. A *process* is a subset of Act^*; $\mathrm{Proc} = \mathcal{P}(\mathrm{Act}^*)$.

Definition 1.12 (Sequential constructors)**.** For $a \in \mathrm{Act}$, $P \in \mathrm{Proc}$, $F : \mathcal{M} \to \mathrm{Proc}$:

$$\begin{aligned} a \cdot P &= \{\varepsilon\} \cup \{a\,t \mid t \in P\}, & \mathbf{0} &= \{\varepsilon\}, \\ \mathsf{out}(c,m).P &= \mathsf{out}(c,m) \cdot P, & \mathsf{ev}(m).P &= \mathsf{ev}(m) \cdot P, \\ \mathsf{in}(c).F &= \textstyle\bigcup_{m \in \mathcal{M}} \mathsf{in}(c,m) \cdot F(m), & \mathsf{new}.F &= \textstyle\bigcup_{k \in \mathbb{N}} \mathsf{new}(k) \cdot \{t \in F(\nu_k) \mid k \notin \mathrm{nonces}(t)\}. \end{aligned}$$

Definition 1.13 (Parallel composition)**.** Let I be a set. For $s \in (I \times \mathrm{Act})^*$ and $i \in I$, $s{\restriction}i \in \mathrm{Act}^*$ is the word of actions of s tagged i, in order, and $\overline{s} \in \mathrm{Act}^*$ is s with tags removed. For $f : I \to \mathrm{Act}^*$, $\mathrm{Interleaves}(f,s)$ holds if $s{\restriction}i = f(i)$ for all i. The relation $\mathrm{comms} \subseteq (I \times \mathrm{Act})^* \times \mathrm{Act}^*$ is the least one such that

$$\begin{aligned} &\mathrm{comms}(\varepsilon,\varepsilon), \\ &\mathrm{comms}(s,t) \Rightarrow \mathrm{comms}\big((i,a)\,s,\ a\,t\big), \\ &\mathrm{comms}(s,t) \wedge i \neq j \Rightarrow \mathrm{comms}\big((i,\mathsf{out}(c,m))\,(j,\mathsf{in}(c,m))\,s,\ \mathsf{comm}(c,m)\,t\big). \end{aligned}$$

For $P : I \to \mathrm{Proc}$,

$$\begin{aligned} \mathrm{merge}_I(P) = \{t \mid \exists f,s.\ (\forall i.\ f(i) \in P(i)) \ \wedge\ &\mathrm{Interleaves}(f,s) \\ &\wedge\ \mathrm{comms}(s,t) \ \wedge\ \mathrm{nonces}(t) \text{ has no repeated letter}\}. \end{aligned}$$

$P \mid Q = \mathrm{merge}_{\{0,1\}}(0 \mapsto P,\ 1 \mapsto Q)$ and $!P = \mathrm{merge}_{\mathbb{N}}(k \mapsto P)$.

Proposition 1.14. *Let* $\mathrm{comms}(s,t)$.

1. $\mathrm{nonces}(t) = \mathrm{nonces}(\overline{s})$.
2. *If a occurs in t then (i,a) occurs in s for some i, or $a = \mathsf{comm}(c,m)$ and $(i,\mathsf{out}(c,m))$, $(j,\mathsf{in}(c,m))$ occur in s for some $i \neq j$.*
3. *If* $\mathrm{Closed}(t)$ *and $(k,\mathsf{in}(c,m))$ occurs in s, then $(i,\mathsf{out}(c,m))$ occurs in s for some $i \neq k$.*
4. $!P = P \mid !P$ *for every* $P \in \mathrm{Proc}$.

1.3 The attacker and the security properties

Definition 1.15 (Enemy)**.** The processes $\mathrm{Enemy}(K) \in \mathrm{Proc}$, for $K \subseteq \mathcal{M}$, are the pointwise least such that

1. $\varepsilon \in \mathrm{Enemy}(K)$;
2. if $K \vdash c$ and $e \in \mathrm{Enemy}(K \cup \{m\})$ then $\mathsf{in}(c,m)\,e \in \mathrm{Enemy}(K)$, for every $m \in \mathcal{M}$;

3. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{out}(c, m)\, e \in \mathrm{Enemy}(K)$;
4. if $K \vdash c$, $K \vdash m$ and $e \in \mathrm{Enemy}(K)$ then $\mathsf{comm}(c, m)\, e \in \mathrm{Enemy}(K)$, an output to itself;
5. if $k \notin \mathrm{nonces}(e)$ and $e \in \mathrm{Enemy}(K \cup \{\nu_k\})$ then $\mathsf{new}(k)\, e \in \mathrm{Enemy}(K)$.

Definition 1.16 (Attacks, knowledge)**.** For $P \in \mathrm{Proc}$,

$$\mathrm{Attacks}(P) = \{t \in P \mid \mathrm{Enemy}(\emptyset) \ : \ \mathrm{Closed}(t)\}.$$

For $K \subseteq \mathcal{M}$, $\mathrm{known}(K, \cdot) : \mathrm{Act}^* \to \mathcal{P}(\mathcal{M})$ is defined by

$$\begin{aligned} \mathrm{known}(K, \varepsilon) &= K, \\ \mathrm{known}(K, a\, t) &= \mathrm{known}\left(\langle K \cup \{m \mid a = \mathsf{comm}(c, m),\ c \in K\}\rangle,\, t\right). \end{aligned}$$

Proposition 1.17 (Prefix-closure)**.** $\mathrm{Enemy}(K)$ *and every process built from the constructors of Definitions 1.12 and 1.13 are prefix-closed. If* P *is prefix-closed, so is* $\mathrm{Attacks}(P)$.

Definition 1.18 (Facts, secrecy, correspondence)**.** A *fact* is $\mathsf{event}(e)$ or $\mathsf{knows}(m)$ with $e, m \in \mathcal{M}$; Fact is the set of facts. A fact *holds* after $t \in \mathrm{Act}^*$, written $t \models f$, if

$$t \models \mathsf{event}(e) \iff \mathsf{ev}(e) \in t, \qquad t \models \mathsf{knows}(m) \iff m \in \mathrm{known}(\langle\emptyset\rangle, t).$$

For a set X, $F, H : (X \to \mathcal{M}) \to \mathrm{Fact}$ and $s \in \mathcal{M}$,

$$\begin{aligned} \mathrm{Corr}(P, F, H) \iff\ & \forall \sigma : X \to \mathcal{M}.\ \forall t \in \mathrm{Attacks}(P). \\ & t \models F(\sigma) \Rightarrow \exists \sigma' : X \to \mathcal{M}.\ F(\sigma') = F(\sigma) \wedge t \models H(\sigma'), \\ \mathrm{Secret}(P, s) \iff\ & \forall t \in \mathrm{Attacks}(P).\ t \not\models \mathsf{knows}(s). \end{aligned}$$

By Proposition 1.17, for prefix-closed P these are properties of every prefix of every attack: $\mathrm{Corr}(P, F, H)$ says that some $H(\sigma')$ holds by the time $F(\sigma)$ does.

2 The calculus of ProVerif

This section gives the syntax of the calculus and restates the semantics of Blanchet's survey (§2.1, §2.4, §2.5.1, §3.1–3.3) for it.

2.1 Syntax

Definition 2.1 (Source syntax)**.** Fix a set of variables $x, y, \ldots$ and a set of names $a, b, c, \ldots$ containing the free names $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. Terms, expressions and processes are generated by

$$\begin{aligned} M, N &::= x \mid a \mid f(M_1, \ldots, M_n), \\ D &::= M \mid h(D_1, \ldots, D_n) \mid \mathsf{fail}, \\ P, Q &::= \mathbf{0} \mid \mathsf{out}(N, M); P \mid \mathsf{in}(N, x); P \mid P \mid Q \mid\, !P \mid \mathsf{new}\ a; P \\ &\quad \mid \mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q \mid \mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q \mid \mathsf{event}(M); P, \end{aligned}$$

where f ranges over constructors and h over constructors and destructors. Types are erased.

Definition 2.2 (Rewrite rules)**.** Fix a set of *may-fail variables* u, disjoint from the variables of Definition 2.1, which are the *ordinary variables*. A *may-fail term* is $U ::= M \mid u \mid \mathsf{fail}$ with M a term without names. Each destructor g of arity n carries a finite ordered list $\mathrm{def}(g)$ of *rewrite rules* $g(U_1, \ldots, U_n) \to U$ between may-fail terms, the variables of U occurring in $U_1, \ldots, U_n$; each constructor f carries the single rule $f(\vec{x}) \to f(\vec{x})$; and every $\mathrm{def}(h)$ ends with $h(\vec{u}) \to \mathsf{fail}$ for distinct may-fail variables $\vec{u}$. A *substitution* maps ordinary variables to terms and may-fail variables to may-fail terms.

Definition 2.3 (Model)**.** A *model* consists of a signature Σ_0 of declared constructors whose nullary symbols include $\mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$, a theory (E, Fr) over Σ_0, a finite set of declared destructors each with its rules as in Definition 2.2 over the signature with data $\Sigma = \Sigma_0 \uplus \mathrm{Data}$ of Definition 1.7, sets $\mathrm{Pub} \subseteq \bigcup_n (\Sigma_0)_n$, Dt of public constructors and public destructors, and a closed process P_0 with $\mathrm{fn}(P_0) \subseteq \mathcal{N}_{\mathrm{pub}} \uplus \mathcal{N}_{\mathrm{priv}}$. The constructors of the calculus are those of Σ, and its destructors the declared ones and the *built-in* ones: the projections $i\mathrm{th}_n$, each with the single rule $i\mathrm{th}_n(\mathsf{tuple}_n(x_1, \ldots, x_n)) \to x_i$, and the boolean destructors equal, not, and and or with the rules

$$\begin{array}{llll} \mathsf{equal}(x,x) \to \mathsf{true}, & \mathsf{equal}(x,y) \to \mathsf{false}, & \mathsf{not}(\mathsf{true}) \to \mathsf{false}, & \mathsf{not}(x) \to \mathsf{true}, \\ \mathsf{and}(\mathsf{true}, u) \to u, & \mathsf{and}(x,u) \to \mathsf{false}, & \mathsf{or}(\mathsf{true}, u) \to \mathsf{true}, & \mathsf{or}(x,u) \to u, \end{array}$$

u a may-fail variable, so that $\mathsf{and}(\mathsf{false}, \mathsf{fail})$ evaluates to false; $M = N$, $M \neq N$, $M \,\&\&\, N$, $M \,||\, N$ and $\mathsf{not}(M)$ abbreviate the expressions $\mathsf{equal}(M,N)$, $\mathsf{not}(\mathsf{equal}(M,N))$, $\mathsf{and}(M,N)$, $\mathsf{or}(M,N)$ and $\mathsf{not}(M)$. The data symbols and the built-in destructors are public, and true in (Red Cond) is the data constant. These are the survey's tuples, projections and booleans (§2.1, (2.5)) and the manual's infix operators (Appendix B), built in as ProVerif has them.

Definition 2.4 (Queries)**.** A *query fact* is $\mathsf{event}(M)$ or $\mathsf{attacker}(M)$ with M a term over constructors, free names and a finite set X of variables; it is an *event fact* or an *attacker fact* accordingly. A *query* is $\mathsf{query}\ \mathsf{attacker}(M)$ with M without variables, or $\mathsf{query}\ F \Longrightarrow H$ with F and H query facts over X.

2.2 Semantics

Names are atoms. An equational theory E is a finite set of equations between terms without names; $=_E$ is the closure of E under reflexivity, symmetry, transitivity, application of function symbols and substitution. The survey states §3 for the core calculus; here its definitions are taken modulo $=_E$: a step on a term $M' =_E M$ counts as a step on M, and substitutions are compared modulo $=_E$.

Evaluation $D \Downarrow U$ of closed expressions to closed may-fail terms is the least relation with $M \Downarrow M$, $\mathsf{fail} \Downarrow \mathsf{fail}$, and $h(D_1, \ldots, D_n) \Downarrow \sigma U'_j$ whenever $D_i \Downarrow U_i$ for all i, $\mathrm{def}(h) = (h(U'_{i,1}, \ldots, U'_{i,n}) \to U'_i)_{i \le k}$, $\sigma U'_{j,i} =_E U_i$ for all i, and for all $i < j$ and all σ' there is some l with $\sigma' U'_{i,l} \neq_E U_l$.

A *configuration* is $(\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}), \mathcal{P}$ with $\mathcal{N}_{\mathrm{pub}}, \mathcal{N}_{\mathrm{priv}}$ finite disjoint sets of names and $\mathcal{P}$ a finite multiset of closed processes with $\mathrm{fn}(\mathcal{P}) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$. Writing $\mathcal{E}$ for the pair of name sets,

reduction is the least relation closed under

$$\mathcal{E}, \mathcal{P} \cup \{\mathbf{0}\} \to \mathcal{E}, \mathcal{P} \qquad \text{(Red Nil)}$$

$$\mathcal{E}, \mathcal{P} \cup \{P \mid Q\} \to \mathcal{E}, \mathcal{P} \cup \{P, Q\} \qquad \text{(Red Par)}$$

$$\mathcal{E}, \mathcal{P} \cup \{!P\} \to \mathcal{E}, \mathcal{P} \cup \{P, !P\} \qquad \text{(Red Repl)}$$

$$(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \mathcal{P} \cup \{\mathsf{new}\ a; P\} \to (\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}} \cup \{a'\}), \mathcal{P} \cup \{P\{a'/a\}\} \quad \text{if } a' \notin \mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}} \qquad \text{(Red Res)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{out}(N, M); Q,\ \mathsf{in}(N', x); P\} \to \mathcal{E}, \mathcal{P} \cup \{Q,\ P\{M/x\}\} \quad \text{if } N =_E N' \qquad \text{(Red I/O}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\{M/x\}\} \quad \text{if } D \Downarrow M \qquad \text{(Red Eval 1)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } D \Downarrow \mathsf{fail} \qquad \text{(Red Eval 2)}$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \quad \text{if } M =_E \mathsf{true} \qquad \text{(Red Cond 1}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q\} \to \mathcal{E}, \mathcal{P} \cup \{Q\} \quad \text{if } M \neq_E \mathsf{true} \qquad \text{(Red Cond 2}')$$

$$\mathcal{E}, \mathcal{P} \cup \{\mathsf{event}(M); P\} \to \mathcal{E}, \mathcal{P} \cup \{P\} \qquad \text{(Red Event)}$$

A *trace* is a finite reduction sequence $\mathcal{E}_0, \mathcal{P}_0 \to \cdots \to \mathcal{E}_n, \mathcal{P}_n$, written $\mathcal{E}_0, \mathcal{P}_0 \to^* \mathcal{E}_n, \mathcal{P}_n$.

Definition 2.5 (Survey, Definitions 3.1–3.5)**.** Let P_0 be a closed process.

1. A closed process Q is an $\mathcal{N}_{\text{pub}}$-*adversary* if $\text{fn}(Q) \subseteq \mathcal{N}_{\text{pub}}$, every function symbol of Q is public, and Q contains no event.
2. A trace *outputs* M *publicly* if it contains a step (Red I/O$'$) whose output is $\mathsf{out}(N, M'); Q$ with $N \in \mathcal{N}_{\text{pub}}$ and $M' =_E M$. It *executes* $\mathsf{event}(M)$ if it contains a step (Red Event) on $\mathsf{event}(M'); P$ with $M' =_E M$.
3. P_0 *preserves the secrecy of* the closed term M *from* $\mathcal{N}_{\text{pub}}$ if, for some $\mathcal{N}_{\text{priv}}$ disjoint from $\mathcal{N}_{\text{pub}}$ with $\text{fn}(M) \cup \text{fn}(P_0) \subseteq \mathcal{N}_{\text{pub}} \cup \mathcal{N}_{\text{priv}}$, no trace $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\} \to^* \mathcal{E}', \mathcal{P}'$ with Q an $\mathcal{N}_{\text{pub}}$-adversary outputs M publicly.
4. P_0 *satisfies* $\mathsf{event}(M) \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} \mathsf{event}(M_{jk})$ *against* $\mathcal{N}_{\text{pub}}$*-adversaries* if, for some $\mathcal{N}_{\text{priv}}$ as above, for every $\mathcal{N}_{\text{pub}}$-adversary Q, every trace Tr from $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\}$ and every substitution σ: if Tr executes $\mathsf{event}(\sigma M)$ then there are σ' and j with $\sigma' M =_E \sigma M$ such that Tr executes $\mathsf{event}(\sigma' M_{jk})$ for every $k \leq l_j$.

Fix $\mathcal{N}_{\text{priv}}$ as in Definition 2.5, and for an $\mathcal{N}_{\text{pub}}$-adversary Q write $\mathcal{C}_Q$ for the configuration $(\mathcal{N}_{\text{pub}}, \mathcal{N}_{\text{priv}}), \{P_0, Q\}$.

Definition 2.6 (Satisfaction)**.** A trace *satisfies* $\mathsf{attacker}(M)$ if it outputs M publicly, and *satisfies* $\mathsf{event}(M)$ if it executes $\mathsf{event}(M)$.

Definition 2.7 (Correspondences between facts)**.** Let F and H_{jk} ($j \leq m$, $k \leq l_j$) be query facts over X. P_0 *satisfies* $F \rightsquigarrow \bigvee_{j=1}^{m} \bigwedge_{k=1}^{l_j} H_{jk}$ *against* $\mathcal{N}_{\text{pub}}$*-adversaries* if for every $\mathcal{N}_{\text{pub}}$-adversary Q, every trace Tr from $\mathcal{C}_Q$ and every substitution σ: if Tr satisfies σF then there are σ' and j with $\sigma' F =_E \sigma F$ such that Tr satisfies $\sigma' H_{jk}$ for every $k \leq l_j$. For event facts this is Definition 2.5(4).

3 The translation

The calculus is translated into the trace semantics: a destructor to a function on may-fail messages, an expression to its value, a process to a set of traces and a query to the property it

denotes. Section 3.2 relates the translation to the semantics of Section 2.

Throughout, fix a model (Definition 2.3) satisfying

(R1) Fr is the set of symbols at the root of no equation and (E, Fr) satisfies (H);

(R2) in every rule of every destructor, the terms among the arguments have all their constructors in Fr;

(R3) Pub consists of the public constructors and the names in $\mathcal{N}_{\mathrm{pub}}$, and Dt of the public declared destructors, so that by Definition 1.7 the attacker has the standard operations besides; a projection denotes the standard projection π_i^n under Definition 3.1, and a boolean destructor is not among Ops, which changes nothing, each of its results being true, false or an argument;

(R4) P_0 is a closed process of Definition 2.1.

These conditions exclude some ProVerif models: an equation with a variable as one side, such as the survey's (2.8), violates (H), and a destructor whose rules match on a symbol at the root of an equation has no counterpart under (R2).

3.1 Destructors, expressions, processes and queries

Definition 3.1 (Destructors from rewrite rules)**.** A *valuation* θ sends ordinary variables to $\mathcal{M}$ and may-fail variables to $\mathcal{M}_\perp$, and gives a may-fail term the value $U\theta \in \mathcal{M}_\perp$: the evaluation $M\theta \in \mathcal{M}$ of a term, $\theta(u)$, and $\perp$ for fail. A destructor g with $\mathrm{def}(g) = (g(U_{i,1}, \ldots, U_{i,n}) \to U_{i,0})_{i \le k}$ denotes the function

$$g(\vec{u}) = U_{i,0}\theta \quad \text{for } i \text{ least such that some } \theta \text{ has } U_{i,j}\theta = u_j \text{ for } j = 1, \ldots, n,$$

which the final rule of $\mathrm{def}(g)$ makes total. By (R2) and Proposition 1.5 at most one θ on the variables of the i-th rule has $U_{i,j}\theta = u_j$ for all j, so g is well defined. A term among the $U_{i,j}$ thus accepts only messages, fail only $\perp$, and a may-fail variable any u_j, which is the survey's two kinds of substitution. When no rule but the final one contains fail or a may-fail variable, every other rule requires messages, so $g = g_0^{\uparrow}$ for the $g_0 : \mathcal{M}^n \to \mathcal{M}_\perp$ that those rules define on messages.

Definition 3.2 (Translation)**.** An *environment* ρ maps variables and names to $\mathcal{M}$; ρ_0 maps each free name a to the constant $a \in \mathcal{M}$. Write $\rho M \in \mathcal{M}$ for the homomorphic extension. Expressions are evaluated in $\mathcal{M}_\perp$:

$$[\![M]\!]\rho = \rho M, \qquad [\![\mathsf{fail}]\!]\rho = \perp, \qquad [\![f(\vec{D})]\!]\rho = f^{\uparrow}([\![\vec{D}]\!]\rho), \qquad [\![g(\vec{D})]\!]\rho = g([\![\vec{D}]\!]\rho)$$

for a constructor f and a destructor g, the latter the function of Definition 3.1. Processes are

translated by

$$[\![\mathbf{0}]\!]\rho = \mathbf{0}, \qquad [\![\mathsf{out}(N,M);P]\!]\rho = \mathsf{out}(\rho N, \rho M).[\![P]\!]\rho,$$
$$[\![\mathsf{event}(M);P]\!]\rho = \mathsf{ev}(\rho M).[\![P]\!]\rho, \qquad [\![\mathsf{in}(N,x);P]\!]\rho = \mathsf{in}(\rho N).\big(m \mapsto [\![P]\!]\rho[x \mapsto m]\big),$$
$$[\![P \mid Q]\!]\rho = [\![P]\!]\rho \mid [\![Q]\!]\rho, \qquad [\![\mathsf{new}\ a;P]\!]\rho = \mathsf{new}.\big(m \mapsto [\![P]\!]\rho[a \mapsto m]\big),$$
$$[\![!P]\!]\rho = ![\![P]\!]\rho, \qquad [\![\mathsf{let}\ x = D\ \mathsf{in}\ P\ \mathsf{else}\ Q]\!]\rho = \begin{cases} [\![P]\!]\rho[x \mapsto m] & [\![D]\!]\rho = m \in \mathcal{M}, \\ [\![Q]\!]\rho & [\![D]\!]\rho = \bot, \end{cases}$$
$$[\![\mathsf{if}\ M\ \mathsf{then}\ P\ \mathsf{else}\ Q]\!]\rho = \begin{cases} [\![P]\!]\rho & \rho M = \mathsf{true}, \\ [\![Q]\!]\rho & \rho M \neq \mathsf{true}. \end{cases}$$

The translation of P_0 is $[\![P_0]\!]\rho_0$. A translated process mentions no ν_k except through new.

Remark 3.3. In the development a process is a program in the continuation monad with answer type Proc, run by $\mathrm{run}(b) = b(\lambda_.\,\mathbf{0})$, with primitives $\mathrm{send}(c,m) = \lambda\kappa.\,\mathsf{out}(c,m).\kappa()$, $\mathrm{recv}(c) = \lambda\kappa.\,\mathsf{in}(c).\kappa$, $\mathrm{fresh} = \lambda\kappa.\,\mathsf{new}.\kappa$, $\mathrm{emit}(m) = \lambda\kappa.\,\mathsf{ev}(m).\kappa()$, $\mathrm{fork} = \lambda\kappa.\,\kappa(\mathrm{tt}) \mid \kappa(\mathrm{ff})$, $\mathrm{repl} = \lambda\kappa.\,!\kappa()$ and $\mathrm{stop} = \lambda_.\,\mathbf{0}$; let and if are case analysis in the metalanguage. The clauses above are the unfolding of that presentation.

Definition 3.4 (Translation of queries)**.** For a valuation $\sigma : X \to \mathcal{M}$ write $\sigma M \in \mathcal{M}$ for the evaluation of M with free names as constants, and let the *instance* of a query fact under σ be $\sigma\,\mathsf{event}(M) = \mathsf{event}(\sigma M)$ and $\sigma\,\mathsf{attacker}(M) = \mathsf{knows}(\sigma M)$. The query $\mathsf{query}\ \mathsf{attacker}(M)$ denotes $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$, and $\mathsf{query}\ F \Longrightarrow H$ denotes $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$, the survey's Definition 3.5 with $=$ read as $=_E$.

3.2 The correspondence

Names of the reduction semantics are the free names together with the created names; terms are interpreted in $\mathcal{M}$ through a bijection ι between the names outside $\mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$, those that (Red Res) may create, and $\mathbb{N}$, extended by $\iota a = a$ for free names, $\iota a' = \nu_{\iota a'}$ for created names, $\iota f(\vec{M}) = f(\iota\vec{M})$ and $\iota\,\mathsf{fail} = \bot$; $\iota M = \iota M'$ exactly when $M =_E M'$. Let ρ_ι be the environment with $\rho_\iota(a) = \iota a$ for every free or created name a.

Lemma 3.5 (Evaluation)**.** *For every closed expression D over free and created names, some U satisfies $D \Downarrow U$, and $D \Downarrow U$ implies $\iota U = [\![D]\!]\rho_\iota$.*

Definition 3.6 (Labelled semantics)**.** A *labelled configuration* is $(U, \mathcal{P})$ with $U \subseteq \mathbb{N}$ finite and $\mathcal{P}$ a finite multiset of closed processes whose names are free names and symbols ν_k. The τ-steps $(U,\mathcal{P}) \xrightarrow{\tau} (U,\mathcal{P}')$ are the steps (Red Nil), (Red Par), (Red Repl), (Red Eval 1), (Red Eval 2), (Red Cond 1′) and (Red Cond 2′), with $=_E$ read on $\mathcal{M}$. The visible steps are

$$(U, \mathcal{P} \cup \{\mathsf{out}(N,M);P\}) \xrightarrow{\mathsf{out}([N],[M])} (U, \mathcal{P} \cup \{P\}),$$
$$(U, \mathcal{P} \cup \{\mathsf{in}(N,x);P\}) \xrightarrow{\mathsf{in}([N],[M])} (U, \mathcal{P} \cup \{P\{M/x\}\}) \quad \text{for every closed term } M,$$
$$(U, \mathcal{P} \cup \{\mathsf{out}(N,M);Q,\ \mathsf{in}(N',x);P\}) \xrightarrow{\mathsf{comm}([N],[M])} (U, \mathcal{P} \cup \{Q,\ P\{M/x\}\}) \quad \text{if } N =_E N',$$
$$(U, \mathcal{P} \cup \{\mathsf{new}\ a;P\}) \xrightarrow{\mathsf{new}(k)} (U \cup \{k\}, \mathcal{P} \cup \{P\{\nu_k/a\}\}) \quad \text{for every } k \notin U,$$
$$(U, \mathcal{P} \cup \{\mathsf{event}(M);P\}) \xrightarrow{\mathsf{ev}([M])} (U, \mathcal{P} \cup \{P\}).$$

The third step is (Red I/O′) between two members of $\mathcal{P}$. It is visible rather than a τ-step because a trace records a completed communication, which is what known reads the attacker's knowledge off. With $\stackrel{\alpha}{\Longrightarrow} = \stackrel{\tau}{\to}^{*}\stackrel{\alpha}{\to}\stackrel{\tau}{\to}^{*}$, the set of *weak traces* is

$$\mathrm{Tr}(U,\mathcal{P}) = \{\alpha_1 \cdots \alpha_n \in \mathrm{Act}^* \mid (U,\mathcal{P}) \stackrel{\alpha_1}{\Longrightarrow} \cdots \stackrel{\alpha_n}{\Longrightarrow}\}.$$

Theorem 3.7 (Adequacy)**.** *For every closed process P whose names are free names and symbols ν_k, $[\![P]\!]\rho = \mathrm{Tr}(\emptyset,\{P\})$, where ρ extends ρ_0 by $\rho(\nu_k) = \nu_k$. For all finite multisets $\mathcal{P},\mathcal{Q}$ of such processes,* $\mathrm{Tr}(\emptyset,\mathcal{P}\uplus\mathcal{Q}) = \mathrm{Tr}(\emptyset,\mathcal{P}) \mid \mathrm{Tr}(\emptyset,\mathcal{Q})$.

Let $\mathcal{A}_0 = \mathrm{Attacks}([\![P_0]\!]\rho_0)$. For a trace Tr from $\mathcal{C}_Q$ let $\mathrm{outputs}(\mathrm{Tr}) \subseteq \mathcal{M}$ be the set of ιM for the terms M that Tr outputs publicly, and for $t \in \mathrm{Act}^*$ say that Tr and t *have the same events* if Tr executes $\mathsf{event}(M)$ exactly when $\mathsf{ev}(\iota M)$ occurs in t.

Theorem 3.8 (Attacks)**.** *Fix an $\mathcal{N}_{\mathrm{pub}}$-adversary Q and a trace* Tr *from $\mathcal{C}_Q$, as well as a trace $t \in \mathcal{A}_0$.*

(E) (i) There is $t' \in \mathcal{A}_0$ with the same events as Tr*.*

(ii) There are an $\mathcal{N}_{\mathrm{pub}}$-adversary Q' and a trace Tr′ *from $\mathcal{C}_{Q'}$ with the same events as t.*

(K) Suppose further $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$.

(i) For every $m \in \mathrm{outputs}(\mathrm{Tr})$ there is t' as in (E)(i) such that $m \in \mathrm{known}(\langle\emptyset\rangle, t')$.

(ii) For every $m \in \mathrm{known}(\langle\emptyset\rangle, t)$ there are Q' and Tr′ *as in (E)(ii) such that $m \in \mathrm{outputs}(\mathrm{Tr}')$.*

Theorem 3.9 (Secrecy)**.** *Let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$, let M be a closed term with $\mathrm{fn}(M) \subseteq \mathcal{N}_{\mathrm{pub}} \cup \mathcal{N}_{\mathrm{priv}}$ and let $s = \rho_0 M$. Then* $\mathrm{Secret}([\![P_0]\!]\rho_0, s)$ *holds if and only if P_0 preserves the secrecy of M from $\mathcal{N}_{\mathrm{pub}}$.*

Remark 3.10. The hypothesis $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ cannot be dropped: when $\mathcal{N}_{\mathrm{pub}} = \emptyset$ no trace outputs anything publicly, so P_0 preserves the secrecy of every M, whereas $\mathrm{Secret}([\![P_0]\!]\rho_0, \rho_0 M)$ fails whenever $\emptyset \vdash \rho_0 M$, for instance for a public constant M.

Theorem 3.11 (Correspondences)**.** *Let F and H be query facts over X such that H is an event fact, and let $\mathcal{N}_{\mathrm{pub}} \neq \emptyset$ if F is an attacker fact. Then P_0 satisfies $F \rightsquigarrow H$ against $\mathcal{N}_{\mathrm{pub}}$-adversaries if and only if* $\mathrm{Corr}([\![P_0]\!]\rho_0,\ \sigma \mapsto \sigma F,\ \sigma \mapsto \sigma H)$.

Remark 3.12. Neither hypothesis can be dropped. When $\mathcal{N}_{\mathrm{pub}} = \emptyset$ and $F = \mathsf{attacker}(M)$, P_0 satisfies the correspondence vacuously, whereas Corr fails as soon as $\emptyset \vdash \sigma M$ for some valuation σ. For $H = \mathsf{attacker}(M)$, Definition 2.6 requires σM itself to be output publicly, whereas $\mathsf{knows}(\sigma M)$ holds as soon as σM is derivable: with the symmetric encryption of the survey's §2.1, $\mathcal{N}_{\mathrm{pub}} = \{c\}$ and $P_0 = \mathsf{out}(c, \mathsf{senc}(s,k)); \mathsf{out}(c,k); \mathsf{event}(e)$, Corr holds for $\mathsf{event}(e) \rightsquigarrow \mathsf{attacker}(s)$ while the correspondence of Definition 2.7 fails. The reading by derivability is ProVerif's.

References

[1] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, October 2016. `../Reference/BlanchetFnTPS16.pdf`, doi:10.1561/3300000004.